SURVEY · OCTOBER 2026

The Software Development Life Cycle in the AI Era

State of the art and best practices across fifteen stages, from planning to retrospectives: how the strongest teams use AI agents and supporting tools to improve quality, speed and the lifecycle itself.

01 Planning
02 Requirements
03 Product design
04 Brainstorms & meetings
05 Sprint planning
06 Ticket creation
07 Development
08 Code review
09 CI/CD
10 QA
11 Regression tests
12 Evaluations
13 Maintenance & bugs
14 On-call
15 Retrospective

Compiled from public research (DORA, METR, Stack Overflow, Faros AI, DX, GitClear, Veracode), engineering papers from Google and Meta, Thoughtworks and Martin Fowler, and practitioner and vendor sources. Vendor figures are flagged as such. Numbered references are on the last pages.

CONTENTS

Contents

Executive summary	3
Glossary of terms	4
What the evidence says: the numbers that matter	6
Cross-cutting principles of high-performing AI-era teams	7
1. Planning	8
2. Requirements gathering	8
3. Product design	10
4. Brainstorms and meetings	10
5. Sprint planning	12
6. Ticket creation	12
7. Development	14
8. Code review	16
9. CI/CD	16
10. QA	18
11. Regression tests	18
12. Evaluations	20
13. Maintenance and bug fixes	21
14. On-call	21
15. Retrospective	23
Maturity model	24
A 90-day adoption roadmap	24
Measuring the impact	24
References	26

OVERVIEW

Executive summary

By late 2026, AI use in software teams is close to universal, but the gains are uneven. Teams that already had strong engineering systems are pulling ahead. Teams without them are producing more code without shipping more value.

90%
of tech professionals use AI at work, median ~2 h/day (DORA 2025)[1]

46%
of developers distrust the accuracy of AI output, up from 31% (Stack Overflow 2025)[3]

+91%
PR review time at high-AI-adoption teams, alongside +98% merged PRs (Faros AI 2025)[4]

45%
of AI-generated code tasks introduced a security flaw (Veracode 2025, vendor)[6]

Seven findings

1. **AI amplifies what is already there.** DORA's 2025 research describes AI as an amplifier that strengthens high-performing organizations and magnifies the problems of struggling ones.[1] Loosely coupled architectures, fast feedback loops, good documentation and a clear AI policy decide whether AI helps.
2. **The bottleneck has moved from writing code to verifying it.** Code generation is cheap. Review, testing and comprehension are now the constraint. Faros AI telemetry shows merged PRs nearly doubling while review time also nearly doubles and company-level delivery stays flat.[4]
3. **Feeling fast is not the same as being fast.** In METR's randomized trial, experienced open-source developers were 19% slower with early-2025 tools while believing they were 20% faster.[2] Teams that care about outcomes measure them, not self-reports.
4. **Specs and context are the new source code.** Leading teams invest in spec-driven development (written requirements, design and tasks before generation) and in context engineering (AGENTS.md or CLAUDE.md files, MCP tools, curated examples) so agents behave predictably.[13][15]
5. **Agents work in parallel and in the background.** Work is moving from autocomplete in the editor to delegated, asynchronous agents that take an issue and return a pull request (Copilot coding agent, Codex cloud, Jules, Claude Code). The engineer's job shifts toward orchestration and review.[21]
6. **Evals are how AI-built products ship safely.** For any feature that contains an LLM, eval suites (code checks plus calibrated LLM-as-judge) act as the specification and as a release gate in CI.[19]
7. **Autonomy is bounded and earned.** In on-call, CI and security work, the consensus is agents that investigate and propose while humans approve. Sandboxing, least privilege and defenses against the "lethal trifecta" of private data, untrusted input and an exfiltration path are required.[17][18]

> **THE ONE-PARAGRAPH PLAYBOOK**
>
> Write down intent before generating anything: specs, acceptance criteria and evals. Give agents rich, versioned context and narrow, sandboxed permissions. Let them run in parallel on well-scoped tasks. Put more effort than ever into automated verification (tests, mutation testing, static analysis, evals, AI first-pass review) so human attention goes to design, risk and business logic. Keep PRs small. Measure throughput, stability and review load together, and treat AI adoption as a change in how the organization works, not a tool purchase.

A note on evidence. Independent, rigorous research on AI in the SDLC is still thin and ages fast. This survey prefers peer-reviewed papers (Meta, Google, METR) and large surveys (DORA, Stack Overflow). It marks figures that come from vendors that sell the relevant tools. Treat every percentage as a snapshot in time.

REFERENCE

Glossary of terms

Terms that come up throughout this survey, many of them coined between 2024 and 2026.

Agentic coding
Software development in which an AI agent plans and carries out multi-step changes (reading files, running commands and tests, editing code) toward a goal, instead of only suggesting completions.

AI-native SDLC
A lifecycle redesigned around AI agents as participants in every stage, as opposed to "AI-assisted", where AI is bolted onto an unchanged process.

AGENTS.md / CLAUDE.md
Repository-level markdown files that give coding agents project conventions, architecture, commands and constraints. AGENTS.md is the emerging cross-tool standard.

AI Capabilities Model (DORA)
DORA's 2025 set of seven foundational practices (such as a clear AI policy, healthy data ecosystems, strong version control and small batches) that amplify AI's positive impact.

AI SRE
AI agents that take part in reliability work: alert triage, correlating telemetry, root-cause hypotheses, runbook execution and postmortem drafting.

Automation bias
The human tendency to over-trust output from automated systems. In code review it shows up as rubber-stamping AI-generated diffs.

Background (asynchronous) agent
A coding agent that runs in a cloud sandbox without the developer watching. It is typically assigned an issue and returns a draft PR.

Bounded autonomy
Letting an agent act on its own only inside explicit limits: typed tools, scoped permissions, approval gates and escalation rules.

Code churn
Code that is reverted or heavily rewritten soon after it is written (GitClear uses two weeks). It is used as a proxy for low-quality or poorly understood code.

Comprehension debt
The growing gap between the amount of code a team owns and the amount any human actually understands. It builds when AI-written code is merged without deep review.

Context engineering
Curating what a model sees (instructions, files, tool outputs, memory, examples) so it has the smallest set of high-signal tokens it needs. It has succeeded "prompt engineering" as the core skill.

Context window / context rot
The token budget a model can attend to. Quality tends to drop as the window fills with irrelevant material, which is why tasks are split up and subagents are used.

DORA metrics
Deployment frequency, lead time for changes, change failure rate and failed-deployment recovery time (formerly MTTR). Rework rate was added more recently.

EARS
Easy Approach to Requirements Syntax: templated requirements such as "WHEN <event> THE SYSTEM SHALL <behavior>". Kiro uses it to make specs machine-checkable.

Equivalent mutant
A code mutation that does not change behavior, so no test can catch it. LLMs are now used to filter these out in mutation testing.

Eval (evaluation)
A repeatable test of an AI system's output quality on a dataset, scored by code assertions, an LLM judge or humans.

Eval-driven development (EDD)
The AI-feature counterpart of TDD: define quality criteria and eval cases first, then iterate on prompts, models and tools until they pass. Every production failure becomes a new case.

Flaky test
A test that passes and fails on the same code. Good practice is to detect and quarantine flaky tests rather than retry until they pass.

Golden dataset
A curated, versioned set of inputs with expected outputs or rubrics, used as the regression baseline for evals.

Guardrails
Runtime or pipeline controls that constrain AI behavior: input and output filters, policy checks, permission boundaries and human approval steps.

Hallucination
Confident output that is not grounded in the input or in reality, such as an invented API, a non-existent package or a made-up fact.

Human-in-the-loop (HITL)
A design in which a person reviews or approves AI actions at defined checkpoints.

Lethal trifecta
Simon Willison's term for the dangerous combination of an agent with (1) access to private data, (2) exposure to untrusted content and (3) a way to communicate externally.

LLM-as-judge
Using a model with a rubric to score another model's output. It must be calibrated against human labels before it is trusted.

MCP (Model Context Protocol)
An open protocol for connecting AI applications to tools and data sources (Jira, GitHub, databases, observability) through standard "servers".

Mutation testing
Injecting small faults ("mutants") into code to check whether tests catch them. It is a stronger quality signal than line coverage.

Productivity paradox (AI)
The observation that individual developers produce more with AI while organization-level delivery metrics barely move, because the bottleneck shifts downstream.

Prompt injection
Malicious instructions hidden in content an agent reads (an issue, web page, log or tool description) that hijack its behavior.

RAG
Retrieval-augmented generation: fetching relevant documents or code into the prompt at query time to ground the model's answer.

Rules file
Persistent instructions for an assistant (.cursor/rules, copilot-instructions.md, AGENTS.md) that are shared across a team and versioned with the code.

Sandboxing
Running agents in isolated environments (container or VM, restricted filesystem and network) so what they *can* do is limited, not just what they are *allowed* to do.

Self-healing tests
UI or end-to-end tests whose broken locators or steps are repaired automatically by an AI agent. Humans should still review the repairs.

Shift-left
Moving quality and security checks earlier in the lifecycle. AI makes it cheap to run review, security scans and tests on every change, and even on every agent step.

Skills / slash commands
Packaged, reusable agent instructions for a recurring workflow (such as "create a ticket" or "triage CI") that capture team process as code.

Spec-driven development (SDD)
Writing a structured specification (requirements, design, task list) before generation, then having agents implement against it with traceability. Tools include GitHub Spec Kit, Kiro, OpenSpec and Tessl.

Subagent
A child agent with its own context window, spawned to do a focused subtask (search, review, test) and report a summary back. It keeps the main context clean.

Tool poisoning
A supply-chain attack that hides malicious instructions in an MCP tool's name, description or schema.

Toil
Manual, repetitive, automatable operational work (Google SRE term). It is a prime target for agents.

Trace
A recorded tree of an AI run (LLM calls, tool calls, inputs and outputs), used to debug agents and build eval datasets. Tools include LangSmith, Langfuse, Arize and Braintrust.

Vibe coding
Andrej Karpathy's term (2025) for building by prompting and accepting AI output with little review. Fine for throwaway prototypes, risky in production.

Vibe prototyping
PMs and designers building interactive, working prototypes from prompts (Figma Make, v0, Lovable, Bolt) instead of static mocks.

Worktree (git)
Multiple checked-out working copies of one repo. This is the standard way to run several agents in parallel without file conflicts.

EVIDENCE BASE

What the evidence says

The most-cited studies of 2024–2026. Read them together: each one measures a different slice, and their tensions are the most useful part.

Study	Type	Key finding	Implication
DORA 2025, State of AI-assisted Software Development[1]	Survey, ~5,000 professionals	90% use AI. AI is now linked to higher delivery throughput but still to instability. AI is an "amplifier". Introduces a 7-practice AI Capabilities Model.	Fix the platform, architecture and feedback loops first, or AI amplifies the dysfunction.
METR RCT (Jul 2025)[2]	Randomized trial, 16 senior OSS devs, 246 tasks	19% slower with AI on mature repos they knew well. Developers believed they were 20% faster. Under 44% of generations were accepted.	Self-reported speedups are unreliable. Gains depend on the task, the codebase and skill with the tools.
Stack Overflow 2025 Developer Survey[3]	Survey, 49k devs	84% use or plan to use AI. 46% distrust its accuracy. 31% use agents. Only 17% say agents improved team collaboration.	Trust is earned through verification tooling, not mandates.
Faros AI Productivity Paradox (2025)[4]	Telemetry, 10k devs, 1,255 teams (vendor)	+21% tasks, +98% merged PRs, +91% review time, larger PRs, more bugs per developer. No company-level gain.	Review and QA capacity must scale with generation.
DX research (2025–26)[14]	Multi-org telemetry and surveys (vendor)	Long-run PR throughput gains are roughly 10–15%. Utilization is above 90%, so the question is now impact.	Measure utilization, impact and cost. Never use AI metrics for individual performance.
GitClear code-quality reports[5]	153M changed lines (vendor)	Churn and copy-paste duplication rose and refactoring ("moved" code) fell as assistants spread.	Watch for maintainability decay. Make refactoring an explicit task.
Veracode GenAI Code Security (2025)[6]	80 tasks × 100+ LLMs (vendor)	45% of tasks produced insecure code. No improvement with model size. XSS and log injection failed in about 86–88% of cases.	SAST, secret scanning and security review must run on every AI-authored change.
Meta TestGen-LLM (FSE 2024)[7]	Industrial deployment	Filtered LLM tests: 75% built, 57% passed reliably, 25% raised coverage. 73% of suggestions accepted by engineers.	"Assured" generation (build, pass, add value) makes AI tests trustworthy.
Meta ACH (FSE 2025)[8]	Industrial deployment	Mutation-guided LLM tests across 10,795 classes. 73% of tests accepted. LLM equivalent-mutant detection reached 0.95 precision and 0.96 recall with preprocessing.	Aim tests at plausible faults, not coverage numbers.
Google code review and migrations[9][10]	Industrial deployment	7.5% of reviewer comments resolved by ML-suggested edits. LLM-driven migrations about 50% faster, with roughly 74% of changed code generated.	Large, repetitive changes (migrations, refactors) give the clearest wins.

HOW TO RECONCILE THE STUDIES

The numbers conflict only on the surface. AI speeds up *authoring*, especially on greenfield, well-scoped or repetitive work and for less familiar codebases. It can slow down experts on complex, familiar code. Downstream, it adds review, testing and security load. Net organizational gain depends on whether that downstream capacity is automated too. This is the central idea behind every stage-specific recommendation that follows.

FOUNDATIONS

Cross-cutting principles

Ten patterns that show up in every stage at the strongest teams.

Principle	What it looks like in practice
1. Intent before generation	Specs, acceptance criteria, test plans and eval cases are written (often drafted with AI) and reviewed *before* code is generated. The spec is the durable artifact. Code is increasingly disposable.
2. Context is infrastructure	AGENTS.md or CLAUDE.md, architecture decision records, glossaries, example folders and MCP connectors to Jira, Confluence, observability and data are versioned, owned and reviewed like code.[13]
3. Verification scales with generation	For every unit of generation capacity added, matching automated verification is added: tests, mutation testing, static analysis, AI first-pass review and evals.
4. Small batches, always	AI makes big diffs cheap to produce and expensive to review. Top teams cap PR size, split agent work into stacked PRs and ship behind feature flags. DORA lists small batches among its AI-amplifying capabilities.[1]
5. Humans own judgment	Agents draft, propose and investigate. Humans decide on architecture, product trade-offs, security-sensitive merges and production actions. "You ship it, you own it" applies whoever typed it.
6. Bounded, sandboxed autonomy	Least-privilege credentials, sandboxed execution, network egress controls, approval gates for irreversible actions, and audit logs of agent actions.[18]
7. Process as code (skills)	Team workflows such as ticket templates, review checklists, release steps and triage runbooks are encoded as reusable agent skills or commands, so quality does not depend on who is prompting.
8. Parallelism by default	Engineers run several agents at once (worktrees, background agents) on independent tasks and switch between them, acting more like tech leads of a small agent team.[21][24]
9. Measure outcomes, not usage	Track DORA metrics plus review time, rework rate, change failure rate and escaped defects. Track AI utilization only as a diagnostic.[14]
10. Keep the craft alive	Deliberate code reading, design reviews, pairing with juniors and "explain this diff" requirements counter comprehension debt and skill atrophy.[11]

01 Planning

Strategy, roadmap, scoping and estimation. AI shortens the path from fuzzy idea to an evidence-backed, scoped plan.

State of the art

Planning is shifting from opinion-driven roadmapping to **evidence synthesis on demand**. LLMs connected over MCP to analytics, CRM, support tickets, sales-call transcripts and the existing backlog can produce a first-pass opportunity map in hours. Agents with read access to the codebase can give **technical feasibility and effort estimates grounded in the real code** ("which services does this touch?"), which used to need a senior engineer's afternoon. Because AI makes building cheap, many teams now plan "**prototype first**": a working spike replaces a slide in the planning discussion.

WHAT THE BEST TEAMS DO

- Keep a living "product context" corpus (strategy docs, personas, metrics definitions, past decisions) that every planning prompt can draw on.
- Use agents to run **codebase impact analysis** for each roadmap candidate: affected modules, risky dependencies, likely migrations.
- Run **pre-mortems with AI as a devil's advocate**, prompting it for failure modes, missing stakeholders and regulatory risks.
- Re-estimate with the AI-era cost curve in mind. Boilerplate, migrations and tests are far cheaper. Ambiguity, integration and verification are not.
- Treat AI-produced roadmaps as drafts. The decision record is written and owned by a human.

PITFALLS

- Plausible-sounding market analysis with fabricated numbers or sources.
- Over-scoping because "AI will make it fast" while ignoring review and QA load.
- Planning from stale context, such as an outdated architecture doc.

TOOLS

- Claude, ChatGPT and Gemini with connectors (Drive, Jira, Confluence, analytics).
- Productboard AI and Dovetail for feedback.
- Coding agents (Claude Code, Codex) in read-only mode for impact analysis.

02 Requirements gathering

Turning user needs into precise, testable requirements, which matter even more now that agents implement exactly what is written.

State of the art

The most important change is **spec-driven development (SDD)**. Tools such as GitHub Spec Kit (constitution → specify → plan → tasks → implement), AWS Kiro (requirements → design → tasks, with requirements in **EARS** notation), OpenSpec, Tessl and BMAD make the spec a first-class, versioned artifact that agents implement against.[15] Upstream, AI **synthesizes user research** from interview transcripts, support tickets and NPS verbatims, extracting themes with quotes that trace back to the source. Practitioners report this cuts synthesis from days to hours.[25]

WHAT THE BEST TEAMS DO

- **Discovery first, AI drafting second.** AI drafts the PRD only after real research. Every requirement links to evidence (a quote, a metric, a ticket).
- Write requirements as **testable acceptance criteria** (Given/When/Then or EARS) so they can become tests and evals directly.
- Ask the AI to **interrogate the spec**: list ambiguities, missing edge cases, conflicting requirements and unstated non-functional requirements (latency, privacy, accessibility, i18n).
- Keep a project **constitution** (non-negotiable architectural and quality rules) that every spec inherits.
- Version specs in the repo next to the code and review them in PRs like code.

PITFALLS

- "Untethered" AI PRDs that read well but reflect no real user signal.
- Spec bloat: 20-page AI specs nobody reads. Keep them short and structured.
- Specs that drift from the code after implementation.

TOOLS

- GitHub Spec Kit, Kiro, OpenSpec, Tessl.
- ChatPRD, Dovetail, BuildBetter, Productboard.
- Gong and Fireflies transcripts piped into synthesis.

03 Product design

UX, UI and system design. The line between mockup and working software is disappearing.

State of the art

Vibe prototyping is mainstream. Thoughtworks' April 2026 Radar puts Figma Make in "Trial" and notes it is widely used by PMs and designers to build user-testable, high-fidelity prototypes.[12] v0, Lovable, Bolt and Claude artifacts generate working React apps from prompts or Figma frames. Figma's own 2026 survey (vendor) reports designers working in code nearly doubling from 21% to 41%. For **system design**, engineers use AI to draft architecture options, sequence diagrams and ADRs, and to red-team designs for scale, failure modes and security.

WHAT THE BEST TEAMS DO

- Test **working prototypes with real users** in the first week. Prototypes replace a large share of static mocks.
- Wire the **design system into AI tools** (tokens, components, Figma Dev Mode MCP) so generated UI matches production components.
- Have AI generate **2–3 alternative architectures with trade-off tables**, then hold a human design review and record an ADR.
- Use AI for **accessibility and edge-case sweeps** (empty, error and loading states, RTL, long strings) on every design.
- Throw prototypes away. Production code is rebuilt from the validated spec, not promoted from the vibe-coded demo.

PITFALLS

- Prototype-to-production leakage: insecure, untested demo code ends up shipping.
- Homogenized "AI-default" UI that ignores brand and design system.
- Skipping architecture review because "the agent picked something".

TOOLS

- Figma Make, Figma Dev Mode MCP, v0, Lovable, Bolt, Claude artifacts.
- Mermaid and diagram generation, ADR templates.
- Uizard and Galileo-style UI generators.

04 Brainstorms and meetings

From "who's taking notes?" to meetings that turn directly into structured, traceable work.

State of the art

AI notetakers (Zoom AI Companion, Google Gemini in Meet, Microsoft Copilot in Teams, Otter, Fireflies, Granola, Spinach) are standard. The frontier is **closing the loop**: tools that detect decisions and action items and push them into Jira or Linear, with human approval, linking existing tickets when they are mentioned.[23] In brainstorms, LLMs serve as **divergence engines** (generating dozens of options, analogies and "what would X do") and as **convergence aids** (clustering sticky notes, scoring ideas against criteria). Miro and FigJam have built-in AI clustering.

WHAT THE BEST TEAMS DO

- **Write first, meet second**: an AI-drafted pre-read (context, options, open questions) goes out before the meeting, and the meeting is spent on decisions.
- Brainstorm in two phases: **silent human ideation first**, then AI expansion. This avoids anchoring everyone on the model's first answer.
- Have notes produce a **decision log** (decision, owner, rationale, date) as well as a summary, and store it where agents can retrieve it later.
- Have AI draft action items into tickets with a **human approval step**.
- Cancel status meetings. Async AI digests of PRs, tickets and incidents replace many standups.

PITFALLS

- Recording without consent, or sensitive discussions in third-party tools. Set a policy.
- Summaries that flatten disagreement into false consensus.
- Action-item spam turning into junk tickets.

TOOLS

- Granola, Otter, Fireflies, Spinach, Fellow, Zoom, Meet and Teams AI.
- Miro AI and FigJam AI for clustering.
- MCP recipes from notes to Jira.

05 Sprint planning

Prioritizing, sizing and committing, now with agent capacity in the plan.

State of the art

AI assistants built into trackers (Atlassian Rovo, Linear agents and Triage, GitHub Projects with Copilot) and standalone tools can **pre-groom the backlog**: find duplicates, flag tickets missing acceptance criteria, propose splits, and draft sprint goals from roadmap and velocity data.[23] The bigger conceptual change is **planning for a mixed human-agent team**. Some tickets are labeled "agent-ready" (well specified, low ambiguity, good test coverage nearby) and handed to background agents. Humans take the ambiguous, cross-cutting and high-risk work, plus review of agent output.

WHAT THE BEST TEAMS DO

- Run an "**agent-readiness**" **triage**: each ticket is tagged human, agent or pair according to spec clarity, blast radius and test coverage.
- **Budget review capacity explicitly.** If agents produce 20 PRs, someone has to review 20 PRs. Plan reviewer hours, not just coding hours.
- Use AI to run a **dependency and risk scan** across committed tickets (shared files, migrations, external API changes).
- Re-baseline velocity after AI adoption, and do not compare story points across the boundary.
- Keep sprint goals outcome-based so agent throughput does not turn into a ticket count.

PITFALLS

- Over-commitment: generation speed is treated as delivery speed.
- Story-point inflation and gaming once AI is in the loop.
- Handing ambiguous tickets to agents and getting confident wrong PRs back.

TOOLS

- Atlassian Rovo, Linear, GitHub Projects, Jira Product Discovery.
- Spinach, Miro AI sprint planning, Automation Anywhere Plan2Jira.
- Custom planning agents over Jira MCP.

06 Ticket creation

The ticket is now a prompt. Its quality directly determines the quality of agent output.

State of the art

When the "assignee" can be Copilot coding agent, Jules (triggered by a label), Codex or Claude Code, **a ticket is literally the task specification an agent executes**.[21] Teams therefore standardize templates (context, goal, acceptance criteria, non-goals, relevant files, test plan, definition of done) and use AI to **draft, lint and enrich tickets**: linking related code, past incidents and similar tickets, and flagging missing information before grooming.

WHAT THE BEST TEAMS DO

- Use a "**ticket quality**" **skill or linter** that scores readiness (clear acceptance criteria, scope, test notes, links) and blocks vague tickets from sprints.
- Include **pointers, not essays**: relevant file paths, an example PR, the failing test and logs. Context beats prose.
- State **non-goals** explicitly. Agents happily over-deliver and touch unrelated code.
- Auto-create bug tickets from Sentry, Datadog or CI failures with stack trace, frequency, suspected commit and repro steps attached.
- Keep one source of truth: the ticket links to the spec, and the PR links to the ticket.

PITFALLS

- AI-padded tickets: long, generic and low in signal.
- Prompt injection through ticket bodies or comments that agents read. Treat them as untrusted input.
- Duplicate tickets from several AI sources.

TOOLS

- Jira and Rovo, Linear, GitHub Issues with issue forms.
- Sentry Seer, Datadog, PagerDuty auto-ticketing.
- Agent skills for ticket creation and grooming.

07 Development

From autocomplete to agent orchestration. This is the stage that has changed most.

State of the art

There are three working modes, and strong engineers move between them:

Mode	Description	Examples	Best for
Inline assist	Completions and inline edits as you type.	Copilot, Cursor Tab, JetBrains AI	Flow-state coding on familiar code
Interactive agent	The agent plans, edits several files, runs tests and iterates in your terminal or IDE while you steer.	Claude Code, Cursor Agent, Codex CLI, Gemini CLI, Windsurf, Kiro	Features, debugging, refactors
Background agent	Asynchronous work in a cloud sandbox, from an issue to a draft PR.	Copilot coding agent, Codex cloud, Jules, Cursor background agents, Devin, Claude Code on the web	Well-specified tickets, migrations, chores, parallel exploration

Productivity at the frontier comes less from model choice than from **context engineering**[13]: layered rules files (global → repo → package), a curated examples folder, MCP connections to docs, tickets and observability, reusable skills and commands, and hooks that run linters and tests automatically after every agent edit. Anthropic's internal report describes teams across engineering *and* non-engineering functions using Claude Code for codebase navigation, debugging, onboarding, test writing and building internal tools.[24]

WHAT THE BEST TEAMS DO

- **Explore → plan → implement → verify.** Ask the agent to read and plan first, review the plan, then implement with tests. Don't let it write code before the plan is agreed.
- **TDD with agents**: write or generate failing tests first, then let the agent iterate until they pass. Tests act as the guardrail against hallucinated success.
- Run **several agents in parallel** in git worktrees or background sandboxes on independent tasks. Use subagents for search and review to keep the main context clean.
- Invest in **fast local feedback** (sub-minute test runs, type checkers, linters) that agents can call. Agents are only as good as the signals they get.
- **Commit and checkpoint often** so a wrong direction can be thrown away cheaply. Generated code is cheap to regenerate.
- Require the author to **explain every line they submit**, and treat "the AI wrote it" as no excuse.
- Point AI at **large mechanical changes** (framework upgrades, API migrations, dead-code removal). Google reports about 50% faster migrations with this approach.[10]

PITFALLS

- **Vibe coding in production** (Thoughtworks "Hold": complacency with AI-generated code). [11]
- Duplicated code, low refactoring, rising churn. [5]
- Hallucinated or typo-squatted packages ("slopsquatting"). Pin and verify dependencies.
- Agents with broad credentials and network access.
- Context overload: huge rules files that dilute signal.

SIGNALS OF MATURITY

- Shared, versioned AGENTS.md or CLAUDE.md in every repo.
- A team skill and command library.
- Pre-approved MCP servers and an allowlist.
- Sandboxed agent runtime by default.
- Telemetry on AI-assisted PR share, rework and defects.

08 Code review

The new bottleneck. AI-assisted review is now standard, and humans focus on what machines miss.

State of the art

AI reviewers (GitHub Copilot code review, CodeRabbit, Greptile, Graphite Diamond, Qodo, Cursor Bugbot, Claude Code review actions) give a **first-pass review on every PR within minutes**. Copilot review grew from zero to overtaking CodeRabbit in PR volume within about seven months of launch, largely through GitHub distribution (vendor-adjacent data).[26] At Google, authors resolve 7.5% of all reviewer comments by applying an ML-suggested edit.[9] Independent comparisons agree on the limits: AI reviewers are good at style, simple bugs, missing tests, null handling and docs, but **consistently miss business-logic errors, auth flaws and race conditions**.[26] Meanwhile review load is rising sharply (+91% review time).[4]

WHAT THE BEST TEAMS DO

- **Layered review**: deterministic checks (lint, types, SAST, secrets), then an AI reviewer, then the author fixes, then a human reviewer focused on design, correctness of intent, security and maintainability.
- Give the AI reviewer **repo-specific rules** (review guidelines, past incident lessons) and tune out noisy categories to keep false positives low.
- **Cap PR size** (for example around 400 lines) and use stacked PRs. Require a human-written "why" in every PR description, plus a note on which parts were AI-generated.
- Use AI to help the *human* reviewer: PR summaries, "explain this diff", risk heat-maps and "what tests cover this?"
- Require **two humans** on security-sensitive paths (auth, payments, PII), whoever wrote the code.
- Track **review latency and review depth** as first-class metrics.

PITFALLS

- Rubber-stamping (automation bias, review fatigue).
- AI reviewing AI with no human in the loop, which creates correlated blind spots.
- Comment noise that teaches developers to ignore the bot.

TOOLS

- Copilot code review, CodeRabbit, Greptile, Graphite, Qodo, Cursor Bugbot.
- Claude Code and Codex GitHub Actions.
- Semgrep, CodeQL, Snyk, SonarQube (often with AI autofix).

09 CI/CD

The pipeline is the safety net that makes high-volume AI changes shippable.

State of the art

Three developments stand out. (1) **AI failure triage**: a separate workflow, triggered when a CI run completes, classifies failures (real regression, flaky test, infrastructure, config), posts a diagnosis with a confidence score on the PR, and can open a fix PR.[20] (2) **Agents in the pipeline**: coding agents run as CI jobs (Claude Code and Codex GitHub Actions, Copilot agent on Actions) for review, auto-fixing lint and type errors, updating snapshots

and writing changelogs. Jules can reportedly auto-fix failing CI checks on its own PRs.[21] (3) **Eval gates**: for LLM features, eval suites run in CI and block merges on quality regressions.[19] DORA's data on AI-linked instability makes **progressive delivery** (feature flags, canaries, automated rollback) even more important.[1]

WHAT THE BEST TEAMS DO

- Keep triage **decoupled** from the signal-producing job, and **never auto-retry flakes away**. Report, rank by flake rate and quarantine with an owner and a ticket.[20]
- Run SAST, dependency, licence and secret scanning on **every** PR. AI-written code fails security checks often.[6]
- Ship everything behind **feature flags**, with canary analysis and automated rollback on SLO breach.
- Give pipeline agents **scoped tokens** (read repo, write to branch only, no production secrets). CI is a prime prompt-injection target.
- Keep pipelines **fast**. Agents iterate against CI, so a 40-minute pipeline wastes both agent and human time.

PITFALLS

- Agents that "fix" CI by weakening or deleting tests. Protect test files and coverage thresholds.
- Leaked secrets through agent logs.
- Untrusted PRs from forks triggering privileged agent workflows.

TOOLS

- GitHub Actions, CircleCI, Buildkite, GitLab Duo.
- Trunk Flaky Tests, Datadog CI Visibility.
- LaunchDarkly, Statsig, Argo Rollouts.

10 QA

Quality engineering moves from writing scripts to designing test strategy and supervising test agents.

State of the art

Playwright 1.56 shipped built-in **test agents**: a *Planner* that explores the app and writes a test plan, a *Generator* that turns it into code, and a *Healer* that repairs broken tests. Through the Playwright MCP server, any coding agent can also drive a real browser.[16] Agentic QA platforms (QA Wolf, Momentic, Mabl, Testim, Applitools for visual AI) promise autonomous exploratory and regression testing. Independent observers caution that "self-healing" has **little independent evidence** behind it beyond locator repair, and that humans should approve heals.[16] At unit level, Meta's **assured LLM test generation** is the reference design: generated tests are kept only if they build, pass repeatedly (to rule out flakiness) and measurably improve coverage. 73% of the resulting suggestions were accepted.[7]

WHAT THE BEST TEAMS DO

- Derive test cases **from acceptance criteria** automatically. Spec, tests and code stay traceable.
- Apply **filters to generated tests**: compiles, passes N times, adds coverage or kills a mutant, and is reviewed by a human.
- Use agents for **exploratory testing** ("try to break checkout"), accessibility audits and cross-browser sweeps, and turn findings into deterministic tests.
- QA engineers act as **quality architects**: risk-based test strategy, test-data management, and owning the test agents' prompts and skills.
- Use **dogfooding plus AI log analysis** to find issues before customers do.

PITFALLS

- Tests that assert the current (buggy) behavior: the AI "learns" the bug.
- Tautological tests that mock everything.
- Healers silently changing what a test checks.

TOOLS

- Playwright agents and Playwright MCP, Cypress.
- QA Wolf, Momentic, Mabl, Applitools.
- Qodo Cover, Diffblue (Java), EvoSuite plus LLM.

11 Regression tests

Making sure that faster change doesn't mean faster breakage.

State of the art

The leading idea is to **target tests at plausible faults, not at coverage**. Meta's ACH system generates a small number of realistic mutants (simulated bugs) for a concern such as privacy, then has an LLM write tests that kill them. An LLM agent filters equivalent mutants (0.95 precision and 0.96 recall with preprocessing), and 277 of the 571 accepted tests would have been thrown away by a coverage-only criterion.[8] Other trends include **AI test impact analysis** (running only the tests affected by a change), **bug-to-regression-test automation** (every fixed bug gets a failing-then-passing test, often agent-written), and visual and snapshot regression with AI-based diffing that ignores rendering noise.

WHAT THE BEST TEAMS DO

- Follow a "**no fix without a test**" **rule**: agent bug-fix PRs must include a test that fails before the fix and passes after.
- Run **mutation testing** on critical modules (payments, auth, privacy) to measure test-suite strength, and generate tests to close gaps.
- Use **test impact analysis** to keep CI fast as suites grow, with a full nightly run as a backstop.
- **Protect the suite**: CODEOWNERS on tests, alerts when tests are deleted or assertions weakened, and coverage ratchets that only go up.
- Maintain **golden datasets** for data pipelines and LLM features (see Evaluations).

PITFALLS

- Suite bloat: thousands of low-value generated tests that slow CI.
- Coverage theatre: high line coverage with weak assertions.
- Flaky AI-written tests eroding trust in the whole suite.

TOOLS

- PIT, Stryker and mutmut for mutation testing.
- Launchable and Datadog Test Impact Analysis.
- Chromatic, Percy and Applitools for visual regression.

12 Evaluations

The testing discipline for non-deterministic systems, and the defining new SDLC stage of the AI era.

State of the art

Any product with an LLM or agent inside needs evals. Unit tests cannot cover open-ended outputs. The practitioner consensus, led by Hamel Husain, Shreya Shankar, Arize, Braintrust and LangSmith, has converged on a clear method.[19]

1. **Error analysis first.** Read real traces, label failures and build a taxonomy of failure modes before writing any evaluator.
2. **Use the cheapest reliable check.** Code assertions for deterministic properties (schema, tool called, latency, cost). LLM-as-judge for semantic qualities (groundedness, helpfulness, tone).
3. **Calibrate judges** against human labels: binary pass/fail with critiques, measured agreement (precision and recall versus humans). An uncalibrated judge is worse than none.
4. **Don't average dimensions** into a single score. Track task success, safety and efficiency separately.
5. **Gate releases** in CI on eval regressions, and run the *same* evaluators online in production for drift.
6. **Close the loop**: every production incident or thumbs-down becomes a new eval case.

WHAT THE BEST TEAMS DO

- Treat the eval suite as the **executable spec** of the AI feature, owned jointly by PM, domain experts and engineers.
- Have domain experts (not only engineers) label data. A "benevolent dictator" for quality decisions helps.
- Version prompts, models, tools and datasets together. Every experiment is reproducible and compared to a baseline.
- Run each eval **several times** and report variance. Single non-deterministic runs mislead.
- Watch for **saturation**: a 100% pass rate means the eval isn't hard enough.
- Audit the eval harness itself (partial runs, misconfigured metrics, judge parse failures) before trusting the numbers.

PITFALLS

- Generic off-the-shelf metrics ("helpfulness 4.2/5") disconnected from real failures.
- Judge drift when the judge model changes.
- Overfitting prompts to a small eval set.
- Evals as a report nobody reads, with no gate.

TOOLS

- LangSmith, Braintrust, Arize Phoenix, Langfuse, W&B Weave.
- promptfoo, DeepEval, Inspect, OpenAI Evals.
- Human labelling: Argilla, Label Studio.

13 Maintenance and bug fixes

Where agents give some of the clearest, lowest-risk return on investment.

State of the art

Maintenance work is well suited to agents: it is well-scoped, verifiable and repetitive. Leading uses include **automated bug fixing from error trackers** (Sentry Seer and similar tools produce root-cause analysis and a proposed fix from a stack trace and the code), **dependency and framework upgrades** (Renovate or Dependabot plus an agent that fixes the breakages), **large-scale migrations** (Google: about 74% of changed code LLM-generated, about 50% faster end to end)[10], **security autofix** (GitHub Copilot Autofix, Snyk, Semgrep Assistant) and **tech-debt sweeps** (dead code, deprecations, type coverage). Legacy modernization is a fast-growing use, with agents reading undocumented COBOL, Java or PHP and producing documentation, tests and incremental rewrites.

WHAT THE BEST TEAMS DO

- Run a **bug-to-PR pipeline**: error tracker → auto-ticket with context → background agent writes the failing test and fix → human review → canary.
- Use "**characterization tests first**" for legacy code: agents write tests that pin current behavior before refactoring.
- Schedule **recurring maintenance agents** (weekly dependency bumps, flaky-test cleanup, docs refresh) with small, reviewable PRs.
- Keep a **tech-debt budget** (for example 20% of capacity) and spend it through agents. The cost of paying down debt has fallen sharply.
- Use agents for **codebase Q&A and onboarding** ("how does billing retry work?"), which reduces dependence on tribal knowledge.

PITFALLS

- Symptom patches (try/except, null checks) that hide the real defect.
- Agent-driven upgrade PRs that pass CI only because coverage is thin.
- Losing human understanding of the system over time (comprehension debt).

TOOLS

- Sentry Seer, Copilot Autofix, Snyk and Semgrep autofix.
- Renovate and Dependabot plus coding agents.
- Moderne/OpenRewrite (deterministic) combined with LLMs.

14 On-call

AI SRE: agents that investigate at 3 a.m. so humans can decide.

State of the art

"AI SRE" products (Rootly, incident.io, PagerDuty Advance, Datadog Bits AI, Resolve.ai, Cleric, Traversal, Grafana Sift, and AWS and Azure ops agents) **start investigating the moment an alert fires**. They correlate metrics, logs, traces and recent deploys, propose root-cause hypotheses with evidence, suggest the right owner, draft status updates and, after the incident, draft timelines and postmortems.[17] The consensus is **bounded**

autonomy: agents handle detection, triage, investigation and *pre-approved* remediations (restart, scale, roll back a flagged deploy) behind typed tools and approval gates. Practitioners note that agents are already good investigators but generally lack the safety controls needed to act on production on their own.[17]

WHAT THE BEST TEAMS DO

- Have the agent post an **evidence pack** (graphs, suspicious deploys, similar past incidents, runbook steps) in the incident channel within minutes of the page.
- Write **runbooks as code** that agents can run step by step. Human approval is required for anything write-side.
- Do **alert hygiene with AI**: cluster noisy alerts, find duplicates and propose SLO-based alerting to cut pager fatigue.
- Expose observability through MCP (Grafana, Datadog, Sentry, cloud logs) to coding agents, so the same agent can go from incident to fix PR.
- Measure MTTD, MTTA, MTTR, investigation time, alert noise and pages per on-call shift, before and after.
- Rehearse with game days that include the AI agent, and test what happens when it is wrong.

PITFALLS

- An agent with production write access becoming the cause of the outage.[17]
- Confident but wrong root-cause analysis anchoring responders.
- Prompt injection through log lines or alert payloads.

TOOLS

- Rootly AI, incident.io, PagerDuty Advance, Datadog Bits AI.
- Resolve.ai, Cleric, Traversal, Grafana Sift, Groundcover.
- Observability MCP servers.

15 Retrospective

Learning loops backed by data, and improving how the team uses AI itself.

State of the art

AI changes retrospectives in two ways. First, **data-backed retros**: agents pull sprint data (Jira, GitHub or GitLab: cycle time, PR size, review latency, rework, escaped bugs, incidents) and arrive with *hypotheses and discussion prompts*, not conclusions.[22] One practitioner who fed six months of retro notes to an LLM found recurring themes the team had missed. Second, **blameless postmortems drafted by AI** from incident timelines and chat, so humans can spend the time on contributing factors and systemic fixes. A new agenda item has also appeared: the "**AI retro**", which asks where agents helped, where they hurt, and which prompts, rules or skills should change.

WHAT THE BEST TEAMS DO

- Open with an **AI-prepared metrics brief** and a trend comparison against previous sprints. Discuss it rather than reading it aloud.
- Run **cross-retro theme mining** each quarter to find recurring issues that never get fixed.
- Turn retro outcomes into **updated context**: edit AGENTS.md, review checklists, skills and eval cases so lessons reach the agents too.
- Track action items to completion, using AI to follow up on them.
- Keep the human core: psychological safety, candour and team ownership cannot be automated.[22]

PITFALLS

- Metrics turned into surveillance or individual performance scoring, which DX explicitly warns against.[14]
- AI summaries that blur root causes.
- Retros that discuss the AI tool but never change the process.

TOOLS

- Rootly and incident.io retrospectives, Jeli.
- Parabol, EasyRetro, Miro AI.
- DX, LinearB, Swarmia, Jellyfish, Faros for metrics.

ASSESSMENT

Maturity model

A five-level view for locating a team and choosing the next step. Most organizations in late 2026 sit between levels 2 and 3.

Level	Development and review	Quality and operations	Process and organization
L1 Ad hoc	Individuals use chat and autocomplete. No shared rules.	Traditional testing. No AI in CI or on-call.	No AI policy. Shadow AI usage.
L2 Assisted	Team-wide IDE assistants. Some agent usage. AI PR summaries.	AI suggests tests. Manual triage.	Policy exists. Usage is measured, impact is not.
L3 Agentic	Interactive agents with shared AGENTS.md. AI first-pass review on every PR. Small-PR norms.	Assured test generation. CI failure triage. AI incident summaries.	Spec templates. Agent-ready ticket triage. DORA, review time and rework tracked.
L4 Orchestrated	Parallel and background agents from tickets to PRs. Skills library. Sandboxed runtime. MCP allowlist.	Mutation-guided tests. Eval gates in CI. AI SRE evidence packs. Bug-to-PR pipeline.	Spec-driven development. Review capacity planned. Agent cost per outcome tracked.
L5 AI-native	Humans define intent and constraints. Agents implement, verify and maintain. Humans review risk, not lines.	Continuous evals in production. Bounded auto-remediation. Self-maintaining test suites with human audit.	The lifecycle itself is improved by agents (retro → context updates). Roles reshaped around judgment.

A 90-day adoption roadmap

Phase	Actions	Exit criteria
Days 0–30 Foundations	Publish an AI usage policy (data, licensing, security). Choose approved tools and MCP servers. Add AGENTS.md or CLAUDE.md to the top repos. Turn on AI first-pass PR review. Enable SAST and secret scanning on all PRs. Baseline DORA metrics, review time, PR size and rework rate.	Baseline dashboard live. More than 70% of engineers using approved tools weekly.
Days 31–60 Workflows	Introduce spec and ticket templates and a ticket-readiness skill. Pilot background agents on "agent-ready" tickets. Add CI failure triage. Adopt the "no fix without a test" rule and generated-test filters. Start evals with error analysis for every LLM feature.	At least 15% of merged PRs agent-authored with rework no worse than baseline. Eval gate on one AI feature.
Days 61–90 Scale and govern	Build a team skills library. Sandbox agents by default. Add PR size limits and stacked PRs. Plan review capacity in sprints. Pilot an AI SRE investigation agent. Hold an AI retro and update context files. Run mutation testing on critical modules.	Lead time down and change failure rate flat or better. On-call MTTR trend visible. Second baseline review complete.

Measuring the impact

Following DX's framework (utilization, impact, cost)[14] and DORA[1], track a balanced set and never one metric alone:

Dimension	Metrics	Watch-outs
Utilization	Weekly active users, share of AI-assisted or agent-authored PRs, agent tasks per engineer	Diagnostic only. Easy to game. Never use for performance review.
Speed	Lead time for changes, PR cycle time, deployment frequency, time to first review	Speed without stability is debt.

Stability and quality	Change failure rate, rework rate, escaped defects, churn, incidents per PR, security findings per PR	Lagging. Review monthly and quarterly.
Flow and bottlenecks	Review latency, PR size distribution, review load per reviewer, CI duration, flake rate	Usually the first place AI gains disappear.
Developer experience	Survey-based satisfaction, perceived friction, trust in AI output, cognitive load	Self-reported speed is unreliable (METR).[2]
Cost and ROI	AI spend per engineer, cost per merged PR or resolved ticket, net hours saved, "agent hourly rate"	Token costs grow quickly with agents. Budget per team.

SOURCES

References

Retrieved October 2026. "(vendor)" marks research published by a company that sells products in the same category.

1. DORA / Google Cloud. *State of AI-assisted Software Development 2025.* dora.dev/research/ai; research.google; Google Cloud blog
2. METR. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity* (Jul 2025). metr.org; summary: getdx.com, the-decoder.com
3. Stack Overflow. *2025 Developer Survey* press release. stackoverflow.co; shiftmag.dev
4. Faros AI. *The AI Productivity Paradox* (2025) and *AI Engineering Report 2026* (vendor). faros.ai/research; faros.ai blog
5. GitClear. *Coding on Copilot / AI Copilot Code Quality* (vendor). Summary: arc.dev
6. Veracode. *2025 GenAI Code Security Report* (vendor). veracode.com
7. Alshahwan et al. *Automated Unit Test Improvement using Large Language Models at Meta*, FSE 2024. arXiv:2402.09171
8. Foster et al. *Mutation-Guided LLM-based Test Generation at Meta*, FSE 2025. arXiv:2501.12862
9. Google Research. *Resolving Code Review Comments with ML*, ICSE-SEIP 2024. research.google/blog
10. Google. *How is Google using AI for internal code migrations?* arXiv:2501.06972
11. Thoughtworks Technology Radar. *Complacency with AI-generated code* (Hold). thoughtworks.com
12. Thoughtworks Technology Radar. *Figma Make* (Trial, Apr 2026). thoughtworks.com
13. Böckeler, B. / Martin Fowler. *Context Engineering for Coding Agents* (Feb 2026). martinfowler.com
14. DX. *AI Measurement Framework* and *Measuring AI code assistants and agents* (vendor). getdx.com; getdx.com/research
15. Spec-driven development: morphllm.com; Medium guide to Kiro, Spec Kit, BMAD; ssojet.com
16. Playwright AI ecosystem and test agents: testdino.com; testguild.com
17. AI SRE and incident response: rootly.com (vendor); augmentcode.com; dzone.com; infoq.com
18. Agent security and the lethal trifecta: endorlabs.com; stride.build; langchain.com
19. Evals: Husain, H. *LLM Evals FAQ* hamel.dev; Arize LLM-as-a-judge in production; practitioner's guide
20. CI failure triage: dev.to tutorial; kilo.ai flaky-test hunter; testmuai.com
21. Background agents: ssojet.com; buildmvpfast.com; morphllm.com (Jules)
22. Retrospectives: *RetroAI++* arXiv:2506.15172; dev.to: six months of retros; rootly.com
23. Meetings to tickets: Spinach (Atlassian Marketplace); Fireflies; Miro AI sprint planning; Automation Anywhere
24. Anthropic. *How Anthropic teams use Claude Code* (Jul 2025). anthropic.com
25. AI-assisted PRDs: cleverx.com; buildbetter.ai
26. AI code review comparisons: kunalganglani.com; PullFlow analysis; aicodereview.cc
27. Agentic SDLC overviews: The Agentic SDLC Handbook; LTM SDLC AI Radar 2026; Allstacks 2026 predictions